Laborator 4: Instrucțiuni de decizie și repetitive. Algoritmi elementari.



Cuprins

1 Obiectivele laboratorului 2
2 Instrucțiuni simple și compuse 2
3 Instrucțiuni de decizie 2
4 Instrucțiuni repetitive 3
5 Algoritmi elementari pentru numere întregi 4
6 Recomandări de implementare și testare 7
7 Probleme 7

1. Obiectivele laboratorului

La finalul laboratorului, studentul trebuie să poată

- să distingă între instrucțiuni simple și instrucțiuni compuse;
- să folosească instrucțiunile de decizie `if`, `if-else` și `switch`;
- să aleagă corect între buclele `while`, `do-while` și `for`;
- să explice efectul instrucțiunilor `break` și `continue`;
- să construiască algoritmi cu bucle și condiții pentru prelucrarea numerelor întregi;
- să determine divizorii unui număr, să testeze primalitatea și să calculeze CMMDC și CMMMC;
- să descompună un număr în factori primi și să prelucreze cifrele unui număr;
- să verifice dacă un număr este palindrom și să justifice corectitudinea algoritmului.

2. Instrucțiuni simple și compuse

O instrucțiune este o comandă executată de program. O instrucțiune simplă se termină, de regulă, cu punct și virgulă. O instrucțiune compusă (bloc) este o succesiune de instrucțiuni încadrate între acolade.

```
int a = 5;
a += 10;

if (a > 10) {
    cout << "a este mai mare decat 10\n";
    cout << a << '\n';
}
```

Blocurile sunt utile atunci când o ramură a unei instrucțiuni de decizie sau corpul unei bucle trebuie să conțină mai multe instrucțiuni. Indentarea consecventă ajută la urmărirea structurii programului.

3. Instrucțiuni de decizie

3.1. Instrucțiunea `if`

Condiția este evaluată ca valoare logică. Dacă este adevărată, se execută corpul lui `if`; altfel, acesta este omis.

```
if (conditie) {
    // instructiuni executate daca este adevarata
}
```

3.2. Instrucțiunile `if-else` și `else if`

```
if (x > 0) {
    cout << "pozitiv\n";
} else if (x < 0) {
    cout << "negativ\n";
} else {
    cout << "zero\n";
}
```

Într-un lanț `if-else if-else`, condițiile sunt evaluate în ordine și se execută prima ramură al cărei test este adevărat. Acoladele sunt recomandate chiar și pentru corpuri formate dintr-o singură instrucțiune. Ramura `else` se asociază cu cel mai apropiat `if` anterior care nu are deja un `else`.

Exemplu: maximul a două numere

```
int a, b;
cin >> a >> b;
if (a > b)
    cout << a << '\n';
else
    cout << b << '\n';
```

Dacă valorile sunt egale, programul afișează valoarea lui `b`, care este egală cu cea a lui `a`.

3.3. Instrucțiunea `switch`

`switch` permite alegerea unei ramuri în funcție de valoarea unei expresii de tip întreg sau caracter. Etichetele `case` trebuie să aibă valori constante. În mod obișnuit, fiecare ramură se încheie cu `break`; în lipsa acestuia, execuția continuă în ramurile următoare (comportament numit *fall-through*).

```
int optiune;
cin >> optiune;
switch (optiune) {
    case 1:
        cout << "Adunare\n";
        break;
    case 2:
        cout << "Scadere\n";
        break;
    case 0:
        cout << "Iesire\n";
        break;
    default:
        cout << "Optiune invalida\n";
}
```

Ramurile pot fi grupate atunci când trebuie să execute aceeași secvență de instrucțiuni.

4. Instrucțiuni repetitive

4.1. Bucla `while` - test inițial

Condiția este verificată înaintea fiecărei execuții. Dacă este falsă de la început, corpul buclei nu se execută niciodată.

```
int i = 1;
while (i <= 5) {
    cout << i << ' ';
    i++;
}
```

4.2. Bucla `do-while` - test final

Corpul se execută cel puțin o dată, deoarece condiția se verifică la final. Sintaxa necesită punct și virgulă după `while(conditie)`.

```
int x;
do {
    cout << "Introduceti un numar pozitiv: ";
    cin >> x;
} while (x <= 0);
```

4.3. Bucla `for`

Este potrivită mai ales atunci când se cunoaște sau se poate exprima ușor numărul de pași. Cele trei expresii sunt inițializarea, condiția și actualizarea contorului.

```
for (int i = 1; i <= 10; i++) {
    cout << i << ' ';
}
```

O buclă poate fi transformată între formele `for` și `while`, dar alegerea trebuie să favorizeze lizibilitatea.

4.4. break și continue

- `break` termină imediat cea mai apropiată buclă sau instrucțiune `switch`.
- `continue` omite restul corpului iterației curente și trece la următoarea iterație. Într-un `for`, se execută mai întâi expresia de actualizare.

Aceste instrucțiuni trebuie folosite cu atenție: modificarea contorului și condiția de oprire trebuie să rămână clare, pentru a evita buclele infinite.

Exemplu: suma numerelor pare până la depășirea unei limite

```
int n, suma = 0;
while (cin >> n) {
    if (n % 2 != 0)
        continue;
    suma += n;
    if (suma > 100)
        break;
}
cout << suma << '\n';
```

Bucla se oprește la eșecul citirii sau după ce suma numerelor pare depășește 100.

5. Algoritmi elementari pentru numere întregi

În exemplele următoare se presupune că valorile de intrare respectă domeniul precizat. Pentru algoritmii care pot produce valori mari, trebuie avut în vedere riscul depășirii capacității tipului întreg.

5.1. Divizorii unui număr

Un număr natural pozitiv d este divizor al lui n dacă restul împărțirii lui n la d este zero, adică $n \bmod d = 0$. Pentru a afișa toți divizorii pozitivi ai lui $n > 0$, putem testa valorile de la 1 la $\sqrt{n}$, deoarece dacă $n = a \cdot b$, cel puțin unul dintre factori este cel mult $\sqrt{n}$.

```
int n;
cin >> n;
if (n <= 0) {
    cout << "n trebuie sa fie pozitiv\n";
} else {
    for (int d = 1; d * d <= n; d++)
        if (n % d == 0)
            cout << d << ' ' << n / d << endl;
}
```

5.2. Numere prime

Un număr prim este un număr natural mai mare decât 1 care are exact doi divizori pozitivi: 1 și el însuși. Numerele 0 și 1 **nu** sunt prime.

```
bool estePrim(int n) {
    if (n < 2)
        return false;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0)
            return false;
    return true;
}
```

Dacă se găsește un divizor d, numărul nu este prim și funcția se poate opri imediat.

5.3. CMMDC – algoritmul lui Euclid

CMMDC-ul a două numere întregi pozitive este cel mai mare divizor comun al lor. Algoritmul lui Euclid folosește relația $\text{cmmdc}(a, b) = \text{cmmdc}(b, a \bmod b)$, până când al doilea operand devine zero.

```cpp
int cmmdc(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

Pentru intrări strict pozitive, rezultatul este pozitiv. Algoritmul este eficient, având complexitate logaritmică în raport cu valorile de intrare.

5.4. CMMMC

Cel mai mic multiplu comun al numerelor pozitive a și b se poate calcula folosind CMMDC:

$$\text{cmmmc}(a, b) = \frac{a}{\text{cmmdc}(a, b)} \cdot b.$$

Împărțirea se face înaintea înmulțirii pentru a reduce riscul de depășire.

```cpp
long long cmmmc(int a, int b) {
    if (a == 0 || b == 0)
        return 0;
    int x = a, y = b;
    while (y != 0) {
        int r = x % y;
        x = y;
        y = r;
    }
    return 1LL * (a / x) * b;
}
```

Pentru această implementare se presupun $a, b \geq 0$, iar rezultatul trebuie să încapă în `long long`.

5.5. Descompunerea în factori primi

Orice număr întreg $n > 1$ se poate scrie ca produs de numere prime. Testăm divizorii în ordine crescătoare și împărțim repetat numărul la fiecare divizor găsit.

```cpp
int n;
cin >> n;
if (n < 2) {
    cout << "Descompunerea este definita aici pentru n >= 2\n";
} else {
    for (int d = 2; d * d <= n; d++) {
        int putere = 0;
        while (n % d == 0) {
            n /= d;
            putere++;
        }
        if (putere > 0) {
            cout << d << '^' << putere << ' ';
        }
    }
    if (n > 1)
        cout << n << "^1";
}
```

De exemplu, $360 = 2^3 \cdot 3^2 \cdot 5$. După testarea tuturor divizorilor până la rădăcina pătrată a valorii rămase, dacă aceasta este mai mare decât 1, ea este primă.

5.6. Prelucrarea cifrelor unui număr

Pentru un număr natural, ultima cifră se obține cu `n % 10`, iar eliminarea ultimei cifre se face cu `n /= 10`. Cifrele sunt procesate de la dreapta la stânga.

Numărul de cifre și suma cifrelor

```
int n;
cin >> n;
int copie = n;
int nrCifre = 0, suma = 0;
if (copie == 0)
    nrCifre = 1;
while (copie > 0) {
    int cifra = copie % 10;
    suma += cifra;
    nrCifre++;
    copie /= 10;
}
cout << nrCifre << ' ' << suma << '\n';
```

Exemplu: pentru $n = 5072$, numărul de cifre este 4, iar suma cifrelor este $5 + 0 + 7 + 2 = 14$. Pentru această variantă se presupune că $n \geq 0$.

5.7. Inversul unui număr

Inversul zecimal se construiește adăugând ultima cifră la dreapta rezultatului deja construit:

$$\text{invers} \leftarrow 10 \cdot \text{invers} + n \bmod 10.$$

```
int n;
cin >> n;
int copie = n;
long long invers = 0;
while (copie > 0) {
    invers = invers * 10 + copie % 10;
    copie /= 10;
}
cout << invers << '\n';
```

De exemplu, inversul lui 1200 este 21, deoarece zerourile de la începutul reprezentării inverse nu se păstrează ca cifre semnificative. Pentru a evita depășirea, rezultatul este de tip `long long`, dar și acest tip are o limită.

5.8. Numere palindrom

Un număr palindrom se citește la fel de la stânga la dreapta și de la dreapta la stânga. Putem inversa numărul și compara rezultatul cu valoarea inițială.

```
int n;
cin >> n;
int copie = n;
long long invers = 0;
while (copie > 0) {
    invers = invers * 10 + copie % 10;
    copie /= 10;
}
if (n >= 0 && invers == n)
    cout << "palindrom\n";
else
    cout << "nu este palindrom\n";
```

Exemple de numere palindrom: 7, 121, 4554. Exemple care nu sunt palindrom: 120, 1234. În această variantă se acceptă numere naturale; pentru valori negative trebuie stabilită explicit convenția dorită.

5.9. Alte prelucrări frecvente

- **Numărarea cifrelor pare:** extrageți fiecare cifră și testați dacă `cifra % 2 == 0`.
- **Cifra maximă:** inițializați maximul cu 0 pentru un număr natural și actualizați-l la fiecare cifră.
- **Suma divizorilor proprii:** parcurgeți divizorii de la 2 la $n - 1$, fără a include numărul însuși.
- **Număr perfect:** un număr pozitiv este perfect dacă suma divizorilor săi pozitivi proprii este egală cu numărul.

6. Recomandări de implementare și testare

- Tratați explicit cazurile-limită: 0, 1, valori negative și valori mici.
- Verificați dacă bucla se termină și dacă variabila folosită în condiție este actualizată.
- Nu folosiți `while (n > 0)` fără să decideți ce trebuie să se întâmple pentru $n = 0$.
- La CMMMC și la inversarea cifrelor, verificați posibilitatea depășirii tipului.
- Testați fiecare algoritm cu un caz obișnuit și cel puțin un caz-limită.

7. Probleme

Rezolvați problemele în C++. Folosiți instrucțiunile de decizie și repetitive potrivite. Pentru fiecare problemă, precizați domeniul de intrare și testați programul pe exemple relevante.

Problema 1. Citiți un număr întreg și afișați dacă este pozitiv, negativ sau zero.

Problema 2. Citiți trei numere întregi distincte și afișați cel mai mare dintre ele.

Problema 3. Citiți un număr între 1 și 7 și afișați ziua săptămânii corespunzătoare folosind `switch`. Pentru orice altă valoare, afișați un mesaj de eroare.

Problema 4. Citiți n și afișați toate numerele pare din intervalul $[0, n]$. Tratați separat cazul $n < 0$.

Problema 5. Citiți n și calculați $n!$, folosind o buclă `for`. Precizați pentru ce valori ale lui n rezultatul poate fi reprezentat în tipul ales.

Problema 6. Citiți numere întregi până la introducerea valorii 0 și afișați suma numerelor nenule citite.

Problema 7. Citiți $n > 0$ și afișați toți divizorii săi pozitivi.

Problema 8. Citiți n și afișați dacă este prim. Tratați corect valorile $n < 2$.

Problema 9. Citiți două numere naturale pozitive și calculați CMMDC-ul folosind algoritmul lui Euclid.

Problema 10. Citiți două numere naturale și calculați CMMMC-ul folosind CMMDC. Tratați cazul în care unul dintre numere este zero.

Problema 11. Citiți $n \geq 2$ și afișați descompunerea sa în factori primi, inclusiv exponenții fiecărui factor.

Problema 12. Citiți un număr natural și afișați numărul de cifre și suma cifrelor sale. Considerați că reprezentarea lui 0 conține o cifră.

Problema 13. Citiți un număr natural și afișați cifrele sale în ordine inversă.

Problema 14. Citiți un număr natural și verificați dacă este palindrom.

Problema 15. Citiți un număr natural și determinați cifra maximă și cifra minimă.

Problema 16. Citiți un număr natural și numărați câte dintre cifrele sale sunt pare.

Problema 17. Citiți $n > 0$ și calculați suma divizorilor proprii. Folosiți rezultatul pentru a verifica dacă numărul este perfect.

Problema 18. Afișați toate numerele prime din intervalul $[2, n]$. Încercați mai întâi o soluție simplă, apoi evitați testele inutile.

Problema 19. Citiți două numere și calculați CMMDC-ul folosind scăderi repetate, apoi comparați metoda cu algoritmul lui Euclid.

Problema 20. Citiți numere până la valoarea 0 și afișați câte numere pozitive și câte numere negative au fost introduse, fără a număra valoarea de oprire.

Problema 21. Folosiți bucle imbricate pentru a afișa un triunghi de caractere `*` cu n rânduri.

Problema 22. Realizați un meniu repetitiv cu `switch`: 1–test de primalitate, 2–CMMDC pentru două numere, 3–afișarea divizorilor, 0–ieșire. Programul se termină numai când se alege 0.

Checklist pentru verificare

- Condițiile din `if` și din bucle acoperă toate cazurile relevante?
- Pentru fiecare buclă, este clar cum se modifică variabilele și când se oprește?
- Pentru numerele prime, sunt tratate separat 0 și 1?
- Pentru cifre, este tratat separat numărul 0?
- Pentru CMMMC, este tratat cazul în care unul dintre numere este 0?
- Sunt evitate depășirile aritmetice acolo unde rezultatele pot crește rapid?

Noțiuni urmărite: instrucțiuni simple și compuse, `if`, `switch`, `while`, `do-while`, `for`, `break`, `continue` și algoritmi elementari.